

Brandenburgische
Technische Universität
Cottbus - Senftenberg

Brandenburg University of Technology Cottbus - Senftenberg
Faculty 1 - Institute for Computer Science
Chair for Computer Engineering
Insert Study Program

Bachelor/Master Thesis

Thesis title

Student name
Immatriculation Number: 123456

Supervisor
Name

1. Examiner
Name

2. Examiner
Name

Cottbus, 4.11.2026

Declaration

The author of this work hereby declares the following:

1. This work was completed independently, without outside help and without the use of tools other than those specified. No automated writing aids or artificial intelligence were used in its creation.
2. Information obtained directly or indirectly from external sources (including electronic sources) is clearly identified as such. Sources used verbatim and in terms of content are cited according to recognized rules of scientific work.
3. The work has not been submitted in this or a similar form (not even in excerpts) as part of another examination at another university and has not been published.
4. The author agrees that the work may be checked using a plagiarism detection tool.
5. Use of AI writing aids (please change as needed): AI writing aids were used exclusively for spell checking, grammar correction, improving formulations, or programming support. All AI-assisted passages are clearly identified and properly cited.

The author is aware that false information and plagiarism can have serious consequences, including the revocation of academic credentials and grades. The author is aware of the responsibility that comes with writing a scientific paper and has completed the work to the best of his/her knowledge and conscience.

Student name Cottbus, 4.11.2026

Acknowledgments

Abstract

Write your abstract here. This is a concise summary of your thesis, providing readers with an overview of the content, objectives, methodology, results, and conclusions. Avoid including information not found in the thesis itself and keep the abstract to approximately 100–300 words. It must consist of a single paragraph.

Abstract

Verfassen Sie hier Ihren Abstract. Dabei handelt es sich um eine prägnante Zusammenfassung Ihrer Abschlussarbeit, die Ihren Lesern einen Überblick über den Inhalt, die Ziele, die Methodik, die Ergebnisse und die Schlussfolgerungen der Arbeit bietet. Vermeiden Sie es, Informationen zu geben, die in der Arbeit selbst nicht auftauchen und halten Sie sich an eine Länge von ca. 100-300 Wörtern. Der Abstract muss aus einem einzigen Absatz bestehen.

Contents

Acknowledgments ii

Abstract - English iii

Abstract - German iv

Contents v

List of Figures vii

List of Tables viii

List of Abbreviations ix

List of Mathematical Symbols xi

1 Introduction 1
- 1.1 Context and Motivation . . . 1
- 1.2 Research Questions and Challenges . . . 4
- 1.3 Research Methodology . . . 5
- 1.4 Contributions . . . 7
- 1.5 Outline and Roadmap . . . 7

2 Background and Related Work 9
- 2.1 The RISC-V Instruction Set Architecture . . . 9
- 2.2 Microarchitectural Foundations & Memory Hierarchy . . . 10
- 2.3 Agile Hardware Construction & Generator Frameworks . . . 15
- 2.4 Open Source SOC Ecosystems . . . 17
- 2.5 Simulation, Emulation, and Evaluation Methodologies . . . 17
- 2.6 Summary and Identified Research Gap . . . 17

3 Methodology 18

4 Evaluation Setup 20

5 Experimental Results 22

6 Conclusions and Future Directions 23

6.1 Contributions 23
6.2 Limitations 23
6.3 Future directions 23

A APPENDIX CHAPTER TITLE **24**

Bibliography **25**

List of Figures

2.1 The Agile Hardware Compilation Flow: From Chisel Generators to Target Verilog. . . 16

3.1 Overview of the proposed framework. 19

List of Tables

4.1 Experimental parameters used in this thesis. 21

List of Abbreviations

AXI4 Advanced eXtensible Interface 4. 14

BOOM Berkeley Out-of-Order Machine. 3

Chisel Constructing Hardware in a Scala Embedded Language. 2

CISC Complex Instruction Set Computer. 2

CSR Control and Status Register. 6

DRAM Dynamic Random-Access Memory. 3

FFTW Fastest Fourier Transform in the West. 3

FIRRTL Flexible Intermediate Representation for RTL. 3

FPGA Field-Programmable Gate Array. 4

HDL Hardware Description Language. 2

HPM Hardware Performance Monitor. 6

IC Integrated Circuit. 1

ILP Instruction-Level Parallelism. 3

IPC Instructions Per Cycle. 7

MSHR Miss Status Holding Register. 3, 4

RISC Reduced Instruction Set Computer. 2

RISC-V Reduced Instruction Set Computer V. 2

ROB Reorder Buffer. 3, 5

SoC System-on-Chip. 2, 3, 13

TDP Thermal Design Power. 1

UART Universal Asynchronous Receiver-Transmitter. 6

List of Mathematical Symbols

α_{vc} alpha value for video coding

ϵ epsilon (a small positive quantity)

Chapter 1

Introduction

1.1 Context and Motivation

The architecture of modern computing is undergoing a profound paradigm shift. For decades, two foundational principles dictated the trajectory of computational hardware development: Moore's law [1] and Dennard scaling [2]. Moore's law observed that the number of transistors packed onto a dense integrated circuit doubles approximately every two years, enabling exponential growth in computing capability. Simultaneously, Dennard scaling posited that as transistor dimensions shrink, their power density remains constant because operating voltage and capacitance scale downward in proportion to physical size, allowing clock frequencies to increase without causing unmanageable thermal dissipation. However, in the mid- to late-2000s, Dennard scaling broke down due to fundamental physical limits, giving rise to the "power wall" and the emergence of "dark silicon" [3, 4]. The power wall describes the critical barrier where subthreshold leakage currents and non-scaling supply voltages cause power density to escalate beyond feasible thermal cooling limits for Integrated Circuits (ICs). Consequently, modern microprocessors can no longer energize all on-chip transistors simultaneously at nominal operating voltages under realistic Thermal Design Power (TDP) constraints—a phenomenon termed dark silicon. In 2011, researchers projected that at the 8 nm technology node, the fraction of unpowered chip area due to dark silicon could reach as high as 50%–80% [5]. Around 2010, the pace of semiconductor advancement had fallen below the threshold which was predicted by Moore's law, and in 2022 NVIDIA's CEO Jensen Huang even went as far as to declare that Moore's law is dead [6].

As the Turing-award winning computer scientists John Hennessy and David Patterson have observed, we have entered a "New Golden Age for Computer Architecture" [7], in which substantial performance and energy-efficiency gains can no longer rely on simply brute-forcing frequency scaling, but must instead stem from architectural specialization, agile design methodologies, and domain-specific co-design between hardware and software.

To appreciate this transition, it is essential to consider the broader evolution of computer architectures. Classical general-purpose computing fundamentally builds upon the von Neumann architecture, which couples a centralized processing unit to unified memory holding both instructions and

data. Over decades, instruction set architectures evolved primarily along two philosophies: Complex Instruction Set Computer (CISC) and Reduced Instruction Set Computer (RISC). CISC architectures, most prominently x86 and its 64-bit successor x86-64, historically dominated personal computing and enterprise servers by providing expressive, variable-length instructions capable of executing complex multi-step memory operations directly. However, preserving backward compatibility across decades of microarchitectural generations introduced substantial hardware decoding overhead, pipeline complexity, and architectural bloat. Conversely, RISC architectures prioritized streamlined execution pipelines with simple, fixed-length instructions operating strictly on registers via load/store semantics. While commercial RISC architectures like ARM achieved market dominance in mobile and embedded systems through energy efficiency, proprietary architectures impose severe licensing fees, restrictive nondisclosure agreements, and legal barriers that prevent academic researchers and emerging hardware vendors from freely customizing processor microarchitectures.

To overcome these barriers, researchers at UC Berkeley developed Reduced Instruction Set Computer V (RISC-V) [8, 9], an open, royalty-free, and modular instruction set architecture designed as a universal standard for academia and industry. Unlike proprietary architectures, RISC-V avoids licensing restrictions and architectural bloat by offering a clean, minimal base integer specification (such as 32-bit RV32I or 64-bit RV64I) alongside standardized, optional modular extensions. For instance, the standard RV64GC profile combines the 64-bit base integer pipeline with integer multiplication and division (`M`), atomic memory operations (`A`), single- and double-precision IEEE 754 floating-point arithmetic (`F` and `D`), and compressed 16-bit instructions (`C`) for reduced code footprint [9]. On top of that, the RISC-V privileged specification establishes a solid foundation for both bare-metal firmware and general-purpose operating systems like Linux by partitioning hardware execution privileges into User mode (U-mode), Supervisor mode (S-mode), and Machine mode (M-mode) [10].

To harness the potential of RISC-V and accelerate agile architectural exploration, researchers at UC Berkeley developed Chipyard [11]. Chipyard is an integrated open-source SoC design, verification, and simulation framework that brings together processors, accelerators, and memory hierarchies into a cohesive ecosystem.

The methodologies by which System-on-Chips (SoCs) and integrated circuits have historically been designed started to hinder development. Classic Hardware Description Languages (HDLs) like VHDL and Verilog which operate on the Register-Transfer-Level struggle with multi-core processor clusters, complex cache coherency interconnects, dynamic memory controllers, and heterogeneous coprocessors—all concepts that are standard in modern hardware development. Traditional HDLs restrict parameterization to simple bus widths via parameters or generic constructs and simple scalar constants. They lack common concepts found in modern programming languages, such as object-oriented abstraction, parametric polymorphism, and functional metaprogramming. This leads to complex systems in Verilog requiring manual wiring, error-prone text preprocessing, or rigid graphical IP integration environments which are difficult to develop and even harder to maintain. For this reason, modern hardware development increasingly takes place in Constructing Hardware in a Scala Embedded Language (Chisel), a Scala embedded language which embeds hardware construction primitives within the high-level, statically typed Scala programming language [12]. Instead of relying on statically written netlists, engineers develop programmatic hardware *generators* that utilize object-oriented principles, parameterized classes, and higher-order functions to instantiate logic

blocks at elaboration time. Written Chisel code is then elaborated into the Flexible Intermediate Representation for RTL (FIRRTL) [11]. FIRRTL is an intermediate representation layer which performs clock-gating insertion, formal verification instrumentation, and target-specific memory mapping before outputting optimized Verilog code. Chipyard removes the need for manual integration overhead of disparate IP blocks by connecting processor tiles through the standardized, open-source **TileLink** bus protocol [13]. Chipyard also contains both the Rocket Chip and the BOOM core generators, which are both evaluated extensively in this thesis:

- **The Rocket Chip Generator**: An in-order, single-issue scalar core implementing a classic 5-stage pipeline (Fetch, Decode/Register Read, Execute, Memory, Writeback) [14]. The Rocket core is exceptionally area- and power-efficient, making it an ideal candidate for deeply embedded microcontrollers and edge nodes. However, because it dispatches and completes instructions strictly in program order, its pipeline is susceptible to stalls resulting from structural hazards, data dependencies, and off-chip memory access latencies.

- **The Berkeley Out-of-Order Machine (Berkeley Out-of-Order Machine (BOOM))**: A synthesized, parameterizable superscalar out-of-order execution processor generator [15, 16]. BOOM implements advanced microarchitectural techniques inspired by modern high-performance processors, including speculative instruction fetch with hardware branch prediction, register renaming to eliminate false register dependencies, an explicit physical register file, an active Reorder Buffer (ROB), out-of-order issue queues, and non-blocking load/store units. BOOM can be scaled across various pipeline widths (1-wide Small BOOM, 2-wide Medium BOOM, and 3-wide Large BOOM), allowing architects to trade off hardware complexity and silicon area for significant extraction of Instruction-Level Parallelism (ILP).

Besides processor core microarchitecture, the overall system performance is heavily constrained by the *memory wall* [17]: the growing disparity between high-speed logic execution and the comparatively high latency of off-chip Dynamic Random-Access Memory (DRAM).
Accessing off-chip DRAM can incur latencies on the order of hundreds of processor clock cycles. In in-order cores such as the Rocket core, an L1 data cache miss leads to the entire pipeline stalling, which instantly halts instruction retirement. Out-of-order cores like BOOM mitigate this penalty by incorporating non-blocking caches backed by Miss Status Holding Registers (MSHRs) [18]. These enable the core to hide substantial portions of memory access latency by tracking multiple in-flight, outstanding cache misses while continuing to execute independent downstream instructions out of order.
Determining how these microarchitectural trade-offs—namely in-order versus out-of-order execution, issue width scaling, L1 cache capacity and associativity, and the presence of an inclusive L2 cache—impact instruction execution throughput requires systematic empirical observation. Evaluating these SoCs demands computationally intensive workloads, such as the Sobel 2D edge detection filter or the Fast Fourier Transform (Fastest Fourier Transform in the West (FFTW)) [19]. These configurations are simulated using the cycle-exact software tool Verilator [20].

1.2 Research Questions and Challenges

Figuring out which trade-offs lead to the best performance leads to a couple of challenges:

1. **Scaling the microarchitecture and exploiting ILP**: Out-of-order superscalar pipelines are built to maximize instructions per cycle by dynamically extracting instruction-level parallelism. However, the actual performance gains achieved over simpler in-order pipelines are heavily modulated by the computational characteristics of the executed workload (e.g., loop nesting depth, memory dependency chains, and branch predictability). Being able to quantify whether the substantial hardware area and power cost that comes with out-of-order hardware is justified for specific embedded kernels remains to be seen.

2. **Memory Hierarchy Sensitivity and Working Set Interactions**: Choosing cache sizes involves complex trade-offs between hit latency, cycle time, silicon footprint, and miss penalties. In workloads where the entire data set fits within the primary data cache (L1 D$), enlarging the cache yields diminishing returns. However, when the data size exceeds the capacity of the L1 cache, the presence of an intermediate L2 cache becomes essential. Being able to differentiate between cache capacity effects and pure computation-bound performance bottlenecks requires an enormous amount of empirical analysis.

3. **Physical Synthesis and Timing Closure on FPGA Targets**: High-level Chisel generators produce abstract structural Verilog netlists. Synthesizing these generated netlists for physical Field-Programmable Gate Array (FPGA) hardware (such as the Xilinx Virtex-7 VC707) requires resolving complex physical constraints, including multi-clock domain generation via Mixed-Mode Clock Managers (MMCM), TileLink-to-AXI4 memory bridges interfacing with external DDR3 SODIMMs, and achieving timing closure at frequencies up to 90 MHz.

4. **Dynamic Latency Hiding vs. Pipeline Stalls**: While in-order architectures stall synchronously upon encountering a memory miss, out-of-order architectures utilize MSHRs and dynamic scheduling to overlap memory fetch latencies with independent computation. However, the precise degree to which out-of-order execution can completely insulate overall runtime from the absence of secondary caching (e.g., removing the L2 cache entirely) under bare-metal conditions requires experimental validation.

To address these challenges systematically, this bachelor thesis formulates four primary research questions:

1. To which degree does scaling the L1 data cache capacity (from 16 KiB to 64 KiB) and integrating a dedicated L2 inclusive cache reduce data cache miss counts and how does this reduction translate into cycle-level execution time for compute-bound versus memory-sensitive kernels?

2. *How do structural differences between in-order single-issue execution (Rocket) and superscalar out-of-order dynamic execution (BOOM v3) influence instruction-level parallelism, instructions per cycle (IPC), and total execution cycles across representative image processing and numerical workloads?*

3. *What are the synthesis trade-offs, FPGA resource utilization footprints (LUTs, FFs, BRAMs, DSPs), and achievable operating clock frequencies when deploying parameterizable Chipyard SoC configurations to a physical Xilinx Virtex-7 VC707 FPGA development board?*

4. *How effectively do dynamic scheduling, speculative execution, and non-blocking caches with MSHRs in out-of-order cores insulate system performance from off-chip memory latencies when the secondary cache is omitted, compared to the direct stall penalties incurred by the in-order Rocket pipeline?*

1.3 Research Methodology

The methodology of my thesis is divided into five cohesive phases, as seen below:

1.3.1 Architectural Design in Chisel

Using the Chipyard framework, a systematic architectural parameter space comprising 24 distinct hardware processor configurations is defined in Scala (`BenchmarkConfigs.scala`). These entail four processor core families:

1. **Rocket**: A 1-wide, 5-stage in-order baseline core.

2. **Small BOOM**: A 1-wide out-of-order core (4-byte fetch, 1-instruction decode, 32-entry ROB).

3. **Medium BOOM**: A 2-wide out-of-order core (4-byte fetch, 2-instruction decode, 64-entry ROB).

4. **Large BOOM**: A 3-wide out-of-order core (8-byte fetch, 3-instruction decode, 96-entry ROB, 128-bit system bus).

Each of these four core families is evaluated across six cache hierarchy configurations:

- **Baseline**: Default core cache settings (16 KiB or 32 KiB L1 D$, 512 KiB L2).
- **SmallL1**: 16 KiB L1 D$ (64 sets, 4 ways) with a 512 KiB inclusive L2 cache.
- **MedL1**: 32 KiB L1 D$ (64 sets, 8 ways) with a 512 KiB inclusive L2 cache.
- **LargeL1**: 64 KiB L1 D$ (64 sets, 16 ways or 128 sets, 8 ways) with a 512 KiB inclusive L2 cache.
- **LargeL2**: Medium/Large L1 D$ combined with an expanded 2048 KiB (2 MiB, 16 ways) inclusive L2 cache.
- **NoL2**: Direct memory interconnect utilizing Chipyard's `WithBroadcastManager`, completely removing the shared L2 cache.

1.3.2 Bare-Metal Software Infrastructure and Custom Runtime

To execute non-trivial computational algorithms without operating system overhead, a custom bare-metal runtime environment is established in C and assembly:

- **Dynamic Memory Management**: Standard POSIX system calls are unsupported in bare-metal Machine mode. In order to support dynamic memory allocation for complex libraries such as FFTW [19], a custom `_sbrk` bump allocator is implemented in `syscalls.c`, allocating contiguous heap memory in DDR3 RAM starting at the linker script's `_end` boundary.
- **Serial Telemetry**: Telemetry output is driven through a memory-mapped 16550-compatible Universal Asynchronous Receiver-Transmitter (UART) controller located at physical address `0x10020000`. A lightweight, re-entrant formatter (`mpaland/printf`) is integrated and operated at a standard baud rate of 115,200 baud to minimize transmission latency during execution.
- **Cycle-Accurate Hardware Performance Telemetry**: Low-overhead inline assembly routines query RISC-V Control and Status Registers (CSRs), specifically `mcycle`, `minstret`, and up to 29 individual Hardware Performance Monitor (HPM) event counters (`mhpmcounter3`–`mhpmcounter31`), capturing exact cycle counts, instruction retirements, branch mispredictions, and cache access/miss events.

1.3.3 Cycle-Exact Software Simulation via Verilator

Hardware configurations written in Chisel are elaborated into FIRRTL and compiled into Verilog netlists. Chipyard's automated build system utilizes **Verilator** [20] to transform the Verilog netlist into a cycle-exact, multithreaded C++ simulation model. Unlike using spike, this provides full signal observability and cycle-accurate performance counting, enabling bit-level functional verification and deterministic performance profiling across all 24 configurations.

1.3.4 Automated Design Space Exploration Pipeline

To orchestrate benchmark runs across dozens of processor configurations systematically, I developed an automated Python test framework comprising `run_edge_detection_benchmark.py`, `boom_dse.py`, and `rocket_fpga_dse.py`. The framework cross-compiles target application binaries, executes multi-run simulation sequences, parses UART serial telemetry, validates numerical checksums, and aggregates microarchitectural statistics into structured JSON datasets to be used for subsequent analysis.

1.3.5 Physical Synthesis and Implementation on FPGA

To validate physical viability beyond simulation, target SoC designs are ported to the **AMD/Xilinx Virtex-7 VC707 Evaluation Kit**. The physical implementation pipeline in AMD Vivado processes generated Verilog netlists, maps clock domains via an MMCM (stepping down the board's 200 MHz differential clock to target frequencies between 50 MHz and 90 MHz), binds top-level pins via physical constraint files (`.xdc`), bridges the TileLink memory subsystem to an AXI4-compatible DDR3

Memory Interface Generator (MIG) controller, and verifies static timing analysis (STA) metrics for bitstream generation.

1.4 Contributions

The major contributions of this bachelor thesis are as follows:

1. **Systematic Formulation of a 24-Configuration RISC-V Exploration Space**: I developed a parameterized suite of modular Chisel configurations in Chipyard, structuring an exhaustive evaluation matrix spanning single-issue in-order cores (Rocket) and superscalar out-of-order cores across three distinct issue widths (Small, Medium, Large BOOM v3) paired with six fine-grained L1/L2 cache variants.

2. **Design and Deployment of a Bare-Metal Profiling Infrastructure**: I implemented a lightweight, bare-metal runtime for RISC-V, featuring dynamic heap bump allocation (`_sbrk`), memory-mapped UART communication (115,200 baud), and non-intrusive CSR-based performance counter extraction capable of executing image processing algorithms (Sobel edge detection), scientific numerical libraries (FFTW), and embedded benchmark suites (Embench-IoT) without having to rely on an underlying operating system.

3. **Comprehensive Analysis of Microarchitectural Scaling**: I evaluated the performance characteristics of all 24 configurations on the Sobel edge detection image processing kernel. Based on empirical cycle-exact simulation measurements, I demonstrated that core issue width exhibits an almost perfect positive correlation with Instructions Per Cycle (IPC) ($r = +0.9889$), and that out-of-order execution achieves a 1.51× speedup over an in-order pipeline at identical 1-wide issue width strictly through dynamic instruction scheduling.

4. **Empirical Characterization of Memory Hierarchy Sensitivity and Latency Tolerance**: I quantified the decoupled effects of L1 and L2 cache sizing. The results demonstrate that while quadrupling L1 D$ capacity from 16 KiB to 64 KiB eliminates 85% to 95% of data cache misses ($r = -0.9135$), its impact on total runtime is negligible ($r = -0.1636$) due to the compute-bound nature of the kernel. Furthermore, empirical measurements show that omitting the L2 cache imposes a measurable penalty (+4,398 cycles, +0.48%) on the in-order Rocket pipeline due to memory stall cycles, whereas BOOM completely masks DRAM latency through its non-blocking MSHR architecture and out-of-order execution.

5. **Automated DSE Toolchain**: I provided a fully automated, reproducible Python-based benchmarking harness for automated configuration compilation, simulation execution, and microarchitectural data aggregation.

1.5 Outline and Roadmap

The thesis is structured as follows:

- **Chapter 2 (Background and Related Work)**: Provides foundational theoretical background on the RISC-V architecture, the Chisel/FIRRTL agile hardware design methodology, and the Chipyard ecosystem. It details the internal microarchitecture of the Rocket and BOOM cores, examines multi-level cache hierarchies and coherency protocols, and reviews related academic work in open-source processor benchmarking and FPGA prototyping.

- **Chapter 3 (System Architecture and Methodology)**: Presents the formal specification of the 24 hardware processor configurations explored in this work. It details the Scala mixin architecture, the TileLink interconnect, the implementation of the bare-metal runtime environment (heap allocation, serial telemetry, and CSR instrumentation), and the automated Python DSE workflow.

- **Chapter 4 (Evaluation Setup and Benchmark Workloads)**: Details the experimental framework, including the mathematical formulation and C implementation of the 2D edge detection algorithm, the Kodak image dataset, the bare-metal FFTW benchmark, the Embench-IoT suite, the cycle-exact Verilator simulation infrastructure, and the AMD Vivado synthesis and implementation setup for the Xilinx Virtex-7 VC707 FPGA.

- **Chapter 5 (Experimental Results and Discussion)**: Delivers a thorough evaluation and analysis of the benchmark results across all the different configurations. It examines IPC scaling, correlation analysis, data cache miss sensitivity, latency hiding capabilities of out-of-order cores, simulation wall-clock throughput, and FPGA synthesis resource utilization.

- **Chapter 6 (Conclusions and Future Directions)**: Summarizes the primary findings and architectural takeaways of this thesis, discusses current limitations, and outlines future avenues for research, including domain-specific hardware accelerators and operating system deployment via FireMarshal.

Chapter 2

Background and Related Work

2.1 The RISC-V Instruction Set Architecture

2.1.1 Modular Base ISA & Extensions (RV64C)

There are a myriad of differences between Risc-V and ARM. The most glaring one ist that the Instruction Set Architecture (ISA) of RISC-V is open while ARM's ISA is closed. Having a closed ISA means that it is proprietary and controlled by specific companies, i.e. the ARM Holdings company, which issues the licences to companies. This leads to reliability but development and research is hindered due to the companys control and licencing cost. Open ISA's are open source, allowing developers to freely develop and customize the architercture to fit their needs. This flexibility has lead to widespread adoption by major tech companies like IBM, Nvidia and Western Digital (WD).

2.1.2 Privilege Architecture & Execution Modes

In order to provide secure isolation between the system and the user on the software level, The RISC-V ISA defines a hierarchical model which consists of three different hardware privilege modes: User Mode (U-mode), Supervisor Mode (S-Mode) and Machine Mode (M-Mode). These modes dictate the scope of instructions that are accessible, the memory regions and control status registers (CSR).

2.1.2.1 Privilege Modes

The highest level of privilege belongs to the machine mode. It is the only mode which is mandatory for every RISC-V implementation. Operations executed in this mode have unrestricted access to the hardware, the physical memory Map and core-level configuration registers. Since M-Mode code interacts with the microarchitecture without the need for memory virtualisation or access restrictions, low-level firmware, hardware initialization routines and baremetal code runs exclusively on this level. The supervisor mode (S-Mode) is designed to host modern operating system kernels, such as Linux. It supports paged virtual memory subsystems (such as Sv39) which are managed via the supervisor adress and translation and protection register (satp for short). The supervisor mode enforces memory isolation across different user tasks while also restricting the operating system from directly modifying core-critical M-mode parameters.

As the name implies, U-Mode is used for the unprivileged user space applications. The processes

2.2 Microarchitectural Foundations & Memory Hierarchy

2.2.1 In-Order vs. Out-of-Order Execution Paradigms

Rocket exhibits the classic 5-stage in-order pipeline: fetch, decode, execute, memory, writeback. Every instruction is broken into these 5 sequential steps.

1. **Fetch** The first step of the pipeline. It retrieves the instruction from memory using the adress in the program counter (PC) and increments the PC afterwards. Memory can be SRAM or an instruction cache. The PC is a register which contains the address that is presented to the instruction memory at the beginning of a clock cycle. While the cycle is running, the instruction is read from memory while the next PC is being calculated. The PC is being calculated by adding the value of 4 to the current PC and then deciding if that new value or that of a branch/jump calculation is to be used. In the original RISC design, every instruction is 4 bytes long.

2. **Decode** Decodes the opcode of the instruction and reads the needed values from the operand file. If the underlying logic of the decoded instruction cannot be executed, the instruction is not passed down the pipeline and thus the pipeline stalls. During a stall, input flip flops won't accept new bits, so no new calculations will take place during this cycle. If the decoded instruction is a jump or a branch, the target address is computed while the register file is read. In the following cycle, the branch condition is computed.

3. **Execute** This stage contains an ALU and a bit shifter. It might also contain a multiple cycle multiplier and divider. The ALU provides Boolean functions such as XOR and performs the addition and subtraction of integers. It also provides status bits which indicate if the result was 0 or if an overflow occcured. Instructions can be classed into three different categories based on latency. Latency describes the time it takes for an instruction to go from start to finish.

    - **Register-Register-Operations:** Includes add, subtract, compare and logical operations. During this step of the pipeline, the two given arguments are passed to a ALU, which spits out the result by the end of the stage. These operations have a single-cycle latency.
    - **Memory Reference:** Includes all the loads from memory. The ALU added the two arguments, which include a register and a constant offset to produce a virtual adress by the end of the cycle. These operations have a two-cycle latency.
    - **Multi cycle instructions:** Includes dividing and multiplying integers and all floating-point operations. These types of operations feed their arguments to the multi-cycle multiply- and divde units. The remainder of the pipeline can continue working while the multiply/divide blocks are busy. In order to keep the writeback stage simple, these blocks write their results to a seperate set of registers.

4. **Memory access:** In this stage, single-cycle latency instructions are simply forwarded to the writeback stage. This guarantees that both one and two cycle latency instructions write their results at the same stage of the pipeline so that only a singular port of the register file is used and is always available.

5. **Writeback:** During writeback, both single-cycle and two-cycle instructions write their results into the register file.

2.2.1.1 Hazards

When the pipeline produces wrong results we call it a hazard.

- **Structural hazards:** A situation in which two instructions try to use the same ressources at the same time. This can be avoided by replicating hardware.

- **Data hazards:** If an instruction attempts to use data before said data is available in the register file, we call that a data hazard.

  - **Operand Forwarding:** If a subsequent instruction needs the result of the one that came before it, the pipeline can stall due to timeconsuming read/write-operations. This can be fixed by bypassing the write/read stage and forwarding the result of the previous instruction to the execute stage of the following instruction. On actual hardware the control unit of the CPU must implement the logic required to catch these dependencies in order to enact the bypass.

  - **Pipeline interlock**

- **Control hazards:** A control hazard occurs when the pipeline can't figure out the next instruction adress which needs to be fetched in the following cycle. This happens because the control flow instruction (i.e. onditional branch, direct jump, indirect jump, exception) has not been decoded or executed. In a processor with a pipeline, fetching occurs at every clock cycle. This can be simply mitigated by waiting until the target adress and branch condition is known. This leads to so called "bubbles" - cycles where nothing is being done (NOP cycles). Since processing power is wasted, this is not a desirable solution. There are multiple hazard mitigation strategies: Static predictions when hardware prediction tables are missing or disabled. *Predict Not-Taken* assumes the sequential fetch (PC + 4). *Backward Taken* assumes that a backward branch will be taken. A backward branch is one that has a taget adress that is lower than its own adress. *Forward Not-Taken* assumes the forward branch will not be taken. The Rocket core mitigates control hazards by using a 28-entry wide branch target buffer (BTB), a 6-entry wide Return Adress Stack (RAS) and a 512-entry wide branch history table (BHT). If the branch target buffer predicts a target during the instruction fetch stage, Rocket redirects the prorgam coutner with minimal delay. Branch mispredictions are detected at the memory stage of the pipeline. Upon detecting a branch misprediction, rocket flushes the pipeline, meaning any and all instructions in the decode and execute stage are discarded, leading to a fixed 3-cycle misprediction penalty. Since BOOM employs a decoupled, out-of-order superscalar pipeline,

control hazard prediction is handled differently. BOOM fields a two-tier predictor consisting of a fast next-line predictor who generates single-cycle predictions and a multiple-stage backing predictor (BTP) who runs a "tagged geometric histroy with loop predictor" algorithm which can override the next-line predictor at stage F4.

2.2.2 Cache Subsystems, Latency Tolerance & MSHRs

2.2.2.1 Memory Wall & The Average Memory Access Time (AMAT) Model

High performance computing is constrained by what is known as the memory hole program [17]. This problem describes the growing difference between core processing frequency and off-chip dynamic dram latency. An arithmetic operation within an execution unit takes only a single cycle to complete but retrieving an operand from off-chip memory causes a latency of tens to hundreds of clock-cycles. In order to mitigate this problem, modern systems rely on multi-level cache hierarchies, non-blocking cache controllers and latency-tolerant execution paradigms.
We can quantify the effectiveness of a memory system by using the average memory acess time (AMAT) model:

$$\text{AMAT} = t_{\text{L1_hit}} + \text{MR}_{\text{L1}} \times (t_{\text{L2_hit}} + \text{MR}_{\text{L2_local}} \times t_{\text{DRAM}}) \tag{2.1}$$

The letter t denotes the acess latency and MR represents the cache miss rate. In order to keep AMAT low cache parameters such as line-size, set-associativity and total storage capacity are scaled up. By increasing associativity and capacity conflict and capacity misses are eliminated, which makes sure that computationally intensive working sets remain in the fast on-chip static RAM and don't need to be moved to off-chip memory. Chipyard usually implements secondary caching by using the SiFive inclusive L2 cache generator.

2.2.2.2 Blocking versus Non-Blocking Caches

Classic scalar processors implement blocking caches. If a blocking cache find a miss on a load operation, the cache controller halts and further memory operations are blocked. This forces the entire pipeline to stall until the line refill completes. Subsequently, even istructions that are independent who target lines which are already present in the cache are barred from proceding.
In order to deal with this issue, non-blocking caches are introduced.[18] These types of caches offer two important microarchitectural capabilities:

1. **Miss-Under-Miss:** The cache maintains multiple concurrent in-flight memory requests to different memory blocks, thus directly exploiting memory-level parallelism across the memory bus.

2. **Hit-Under-Miss:** The cache controller cobtinues to service cache hits for downstream instructions while an outstanding miss transaction is asynchronously fetched from deeper memory tiers.

2.2.2.3 Miss Status Holding Registers (MSHRs)

The core foundation of a non-blocking cache is rooted ib the mshr file. Each miss status holding register is a specialized piece of hardware that tracks entries that store the metadata of an active, outstanding cache refill: When a load misses in the cache tags and no request for that specific block exists, a free mshr is allocated. The mshr records the block tag, destination physical register or reorder buffer tag and byte offset before dispatching an acquisition transaction across the system. This process is called *primary miss processing*. If subseqeuent instructions reference the same in-flight cache line before the refill returns, the controller avoids issuing redundant bus rtansactions by instead merging the secondary miss into the existing replay queue of the mshr. Upon data return, every pending request within the queue are satisfied with a singular transaction. This process is called *secondary miss merging*. If all the allocated mshr's are busy, the cache controller registers a structural hazard and must subsequently stall or reject further incoming memory requests until an active transaction retires.

2.2.2.4 Latency Tolerance in Rocket versus BOOM

The capability of using non-blocking caches to their full potential is always limited by how processors handle executions. The in-order Rocket core uses a baseline blocking L1 data cache. Even if it is linked to a non-blocking interface, Rockets in-order pipeline logic forces a pipeline interlock the moment a subseqeuent instruction needs the load operand. Long off-chip memory latency means idle pipeline cycles.

2.2.3 On Chip Interconects (TileLink & AXi4)

As system-on-chip SoC complexity scales upwards the Interconect on the chip becomes a vocal factor when regarding memory bandwidth, latency, and system composability. When it comes to modern heterogenous architecture where in-order cores like Rocket are combined with out-of-order superscalar pipelins like BOOM, cache controllers and DMA peripherals, the interconnect needs to guarantee deadlock-free packet transportation while preserving cache coherency. Chipyard provides two complementary bus protocols: The industry-standard AXI4 interface and the native, cache-coherent TileLink.

2.2.3.1 The TileLink Interconnect Protocol

TileLink is a bus protocol which was specifially designed to support the integration of high performance RISC-V soc. [13] TileLink is different from other bus protocols that rely on ad-hoc sideband wiring for multiprocessor synchronization because it is mathematically structured around a directed acyclic graph channel model in order to guarantee deadlock freedom. TileLink defines five independent, decoupled communication channels from A to E:

1. **Channel A Master → Slave:** This channel transmits operation requests like reads (i.e. Get), writes such as *PutFullData*, *PutPartialData* and cache permission acquisitions like *AcquireBlock* and *AcquirePerm*.

2. **Channel B Slave → Master:** Carries coherence probe requests from outer cache managers to interrogate or downgrade lines held within private L1 caches.

3. **Channel C Master → Slave:** transmits probe acknowledgements like *ProbeAck* and *ProbeAck-Data* or voluntary writeback evictions (*Release*, *ReleaseData*)

4. **Channel D Slave → Master:** Returns read data bursts and transaction acknowledgements.

5. **Channel E Master → Slave:** Delivers a single-wire final grant acknowledgement from the master, thus resolving serialization races between concurrent requesters without a centralized bus arbitration.

2.2.3.2 The AXI4 Protocol

While TileLink is responsible for internal coherent traffic, the Advanced eXtensible Interface 4 Advanced eXtensible Interface 4 (AXI4) which is part of the ARM AMBA4 specification is the industrial standard for peripheral and off-chip memory integration. [21] AXI4 seperates read and write operations into five different unidirectional channels: *Read Adress (AR)* and *Read Data (R)* provide pipelined read transactions and data streaming back towards the master. *Write Adress (AW)*, *Write Data (W)* and Write Response (B) are responsible for decouplin adress issuance, payload transmission and final completion handshaking.
AXI4 provides native full-duplex communication due to write- and read data paths posessing dedicated physical signals. Out-of-order transaction completion is supported via transaction id tags (ARID, AWID, RID, BID) which enable memory controllers to reorder memory requests in order to maximize DRAM row-buffer hit rates.

2.2.3.3 SOC integration and protocol bridging within chipyard

Inside the chipyard ecosystem processors, memory systems and crossbars are composed by using the diplomacy framework. [11] This framework negotiates memory-mapped adress ranges, bus widths and transaction ID allotments at the elaboration phase, which eliminates manual wiring bugs and bus-width mismatch hazards.
The system hierarchy is organized as follows:

- Processor tiles (i.e. Rocket and) interface directly with the system bus via TL-C.

- traffic which targets external memory routes across the memory bus to a coherent L2 cache or a TileLink Broadcast manager (if the L2 is ommitted).

- In order to interface with external memory hardware chipyard synthesizes a TileLink to AXI4 protocol bridge. This middleman comverts TileLink channel operations into adress/data beats while simultanously mapping transaction IDS, which ensures the complete interoperability between open-source processors and proprietary memory controllers.

2.3 Agile Hardware Construction & Generator Frameworks

In the past, hardware development relied on waterfall models, manual register-transfer-level (Register-Transfer Level (RTL)) tinkering and static netlists. With increasing complexity, these methods became more and more prone to errors and made debugging harder. In order to mitigate these issues, modern computer architecture design has adopted agile hardware development. [**?**] This is a development paradigm which was adopted from software engineering. It emphasizes iterative tapeouts, continous integration, extensive parameterization and programmatic hardware generators. Instead of having to design fixed, singular hardware instances, developers construct parameterized software programs that emit optimized hardware netlists which are suited for specific application requirements.

2.3.1 Classical RTL (Verilog/VHDL) vs Metaprogrammed Generators (Chisel/FIRRTL)

For the longest time, digital hardware design has been dominated by HDL like Verilog and VHDL. While these languages are great for describing concrete gate-level and cycle-by-cycle register-transfer operations, they aren't suited for modern hardware development.

2.3.1.1 Limitations of Hardware Description Languages

1. **Primitive Parameterization:** Verilog restris parameterization to compile-time scalar constants and static structural instantiation loops. Complex parameter dependencies, such as dynamically sizing crossbar arbitration logic based on the number of heterogenous cache ports or computing routing tables can't be evaluated natively within Verilog.

2. **Lack of abstraction:** HDLslack object-oriented object-oriented inheritance, parametric polymorphism, functional closures and higher-order-functions . If one wants to reuse structures, such as a parameterized queue or a generic arbiter, he would have to reqrite them or duplicate them by hand for every distinct payload.

3. **Fragility of Structural Interconnects:** Building multi-core SoCs in a hardware description language like Verilog requires explicitly wiring tens of thousands of seperate port signals across hierarchical module boundaries. Manual changes across every intermediate netlist is needed if a sideband coherence signal is to be added or if an adress bus needs to be widened. This means a lot of maintenance overhead and manual work, which is very timeconsuming and prone to errors. Engineers rely on fragile helper scripts, often written in Python or Pearl, which lack semantic awareness of the underlying hardware types and need to be maintained on top of the already existing code.

2.3.1.2 Chisel

In order to rise above these architectural limitations, scientists at the university UC Berkely have developed Chisel (**C**onstructing **H**ardware **i**n a **S**cala **E**mbedded **L**anguage). [12] Chisel is a domain specific language embedded inside the modern statically typed Scala programming language. It is

not a high-level synthesis tool that infers hardware from C code. Chisel does not simulate hardware. It constructs a hardware elaboration graph in memory. By executing it in the runtime environment (JVM), hardware developers gain acess to modern software engineering design methods during the hardware elaboration process, such as:

- **Object-Oriented Abstraction:** functional blocks inheretr from abstract classes and provide reusable traits (mixins). Rocket and tiles share common reusable traits like *HasCoreParameters* and *HasL1HellaCacheParameters*, which enable reusing code across different distinct execution pipelines.
- **Higher-Order-Functions and Metaprogramming:** Combinational and repetitive hardware logic can be expressed in a concive way by using functional programming methods like *map*, *reduce* and pattern matching. For example a priority arbiter or an ALU operand decoder that would require a lot of nested lines of code in Verilog can be produced by a single functional pipeline in .
- **Parametric Polymorphism:** Harware modules can be generic regarding data types. For example, a single First-In-First-Out (FIFO) buffer class (`Queue[T <: Data]`) can instatiate queues for single-bit flags, 64-bit integer values or even complex TileLink coherence message bundles without having to change the underlying control logic.
- **Graph-Based Diplomatic Interconnects:** The diplomacy framework models on-chip communication as a directed graph. During an early elaboration pass, connected nodes negotiate bus widths, adress spaces, burst lengths and master/slave ID allocations. These nodes automatically generate the required width adapters, FIFOFixers and crossbar arbiters prior to hardware generation.

2.3.1.3 FIRRTL: The Layer Inbetween

In the Chisel environment, the executing Scala program elaborates the circuit into the (**F**lexible **I**ntermediate **R**epresentation for **RTL**) [11]

2.3.2 The Chipyard Ecosystem & Integrated Generators

2.4 Open Source SOC Ecosystems

2.4.1 Survey of Frameworks (Chipyard, PULP/PULPino, OpenTitan)

2.4.2 Comparative Analysis & Ecosystem Taxonomy

2.5 Simulation, Emulation, and Evaluation Methodologies

2.5.1 Cycle-Exact Simulation (Verilator) vs ISA Emulation (Spike/QEMU)

2.6 Summary and Identified Research Gap

Chapter 3

Methodology

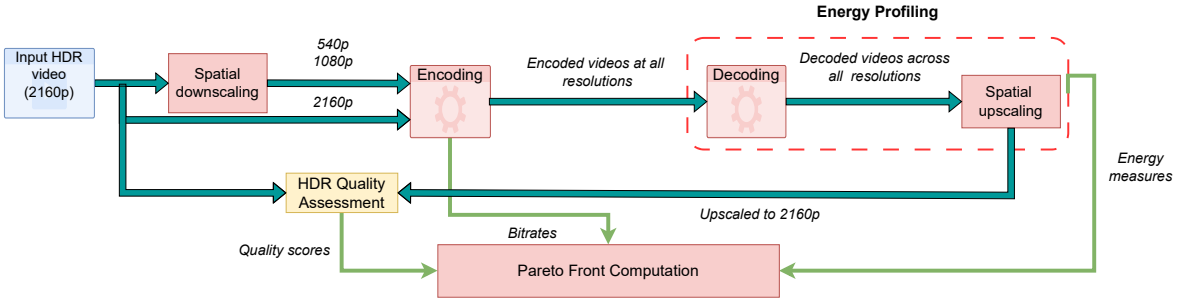


FIGURE 3.1: Overview of the proposed framework.

Chapter 4

Evaluation Setup

TABLE 4.1: Experimental parameters used in this thesis.

Parameter		*Values*					
$\mathcal{R}$	*r [pixels]*	360	432	540	540	540	720
	b [Mbps]	0.145	0.300	0.600	0.900	1.600	2.400
	r [pixels]	720	1080	1080	1440	2160	2160
	b [Mbps]	3.400	4.500	5.800	8.100	11.600	16.800
$\mathcal{C}$		{YUV420, YUV422, YUV444 }					
$\mathcal{D}$		8, 10					
Encoder		*x265 v3.6 [veryslow]*					
Decoder		*HM*					

Chapter 5

Experimental Results

Chapter 6

Conclusions and Future Directions

6.1 Contributions

6.2 Limitations

6.3 Future directions

Appendix A

APPENDIX CHAPTER TITLE

Write your Appendix content here.

Bibliography

[1] G. E. Moore, "Cramming More Components onto Integrated Circuits," *Electronics*, vol. 38, no. 8, pp. 114–117, 1965.

[2] R. H. Dennard, F. H. Gaensslen, H.-N. Yu, V. L. Rideout, E. Bassous, and A. R. LeBlanc, "Design of Ion-Implanted MOSFET's with Very Small Physical Dimensions for Digital Integrated Circuits," *IEEE Journal of Solid-State Circuits*, vol. 9, no. 5, pp. 256–268, 1974.

[3] S. Borkar, "Design Challenges of Technology Scaling," *IEEE Micro*, vol. 19, no. 4, pp. 23–29, 1999.

[4] M. B. Taylor, "A Landscape of the Dark Silicon Problem," *IEEE Micro*, vol. 32, no. 3, pp. 8–14, 2012.

[5] H. Esmaeilzadeh, E. Blem, R. St. Amant, K. Sankaralingam, and D. Burger, "Dark Silicon and the End of Multicore Scaling," in *Proceedings of the 38th Annual International Symposium on Computer Architecture (ISCA)*, 2011, pp. 365–376.

[6] J. Huang, "NVIDIA GTC 2022 Keynote Q&A: Moore's Law is Dead," Keynote Press Briefing, NVIDIA Corporation, 2022.

[7] J. L. Hennessy and D. A. Patterson, "A New Golden Age for Computer Architecture," *Communications of the ACM*, vol. 62, no. 2, pp. 48–60, 2019.

[8] D. A. Patterson and A. Waterman, "The Case for RISC-V," EECS Department, University of California, Berkeley, Tech. Rep. UCB/EECS-2014-146, 2014.

[9] A. Waterman and K. Asanović, "The RISC-V Instruction Set Manual, Volume I: User-Level ISA, Document Version 2.2," RISC-V Foundation, Tech. Rep., 2019.

[10] ——, "The RISC-V Instruction Set Manual, Volume II: Privileged Architecture, Version 1.11," RISC-V Foundation, Tech. Rep., 2019.

[11] A. Amid, D. Biancolin, A. Gonzalez, D. Grubb, S. Karandikar, H. Liew, A. Magyar, H. Mao, J. Ou, N. Pemberton *et al.*, "Chipyard: Integrated Agile RISC-V SoC Design Environment," *IEEE Micro*, vol. 40, no. 4, pp. 35–45, 2020.

[12] J. Bachrach, H. Vo, B. Richards, Y. Lee, A. Waterman, R. Avižienis, J. Wawrzynek, and K. Asanović, "Chisel: Constructing Hardware in a Scala Embedded Language," in *Proceedings of the 49th Annual Design Automation Conference (DAC)*, 2012, pp. 1216–1225.

[13] H. Cook, W. Terpstra, and Y. Lee, "SiFive TileLink Specification: Version 1.7.1," SiFive, Inc., Tech. Rep., 2017.

[14] K. Asanović, R. Avižienis, J. Bachrach, S. Beamer, D. Biancolin, C. Celio, H. Cook, D. Ditzel, B. Elies, R. Gimenez *et al.*, "The Rocket Chip Generator," EECS Department, University of California, Berkeley, Tech. Rep. UCB/EECS-2016-17, 2016.

[15] C. Celio, P.-F. Chiu, B. Nikolić, D. Patterson, and K. Asanović, "BOOM: The Berkeley Out-of-Order Machine," EECS Department, University of California, Berkeley, Tech. Rep. UCB/EECS-2015-167, 2015.

[16] ——, "The Berkeley Out-of-Order Machine (BOOM): An Industry-Competitive, Open-Source RISC-V Processor," *IEEE Micro*, vol. 39, no. 5, pp. 68–75, 2019.

[17] W. A. Wulf and S. A. McKee, "Hitting the Memory Wall: Implications of the Obvious," *ACM SIGARCH Computer Architecture News*, vol. 23, no. 1, pp. 20–24, 1995.

[18] D. Kroft, "Lockup-Free Instruction Fetch/Prefetch Cache Organization," in *Proceedings of the 8th Annual Symposium on Computer Architecture (ISCA)*, 1981, pp. 81–87.

[19] M. Frigo and S. G. Johnson, "The Design and Implementation of FFTW3," *Proceedings of the IEEE*, vol. 93, no. 2, pp. 216–231, 2005.

[20] W. Snyder, "Verilator: Fast Open-Source C++ HDL Simulator," *IEEE Design & Test of Computers*, 2004.

[21] *AMBA AXI and ACE Protocol Specification: AXI3, AXI4, and AXI4-Lite, ACE and ACE-Lite*, Ihi0022e ed., ARM Limited, 2011.

[22] S. Karandikar, H. Mao, D. Kim, D. Biancolin, A. Amid, A. Lee, N. Pemberton, E. Amarel, C. Rigge, K. Asanović *et al.*, "FireSim: FPGA-Accelerated Cycle-Exact Scale-Out System Simulation in the Public Cloud," in *Proceedings of the 45th Annual International Symposium on Computer Architecture (ISCA)*, 2018, pp. 29–42.

[23] A. Traber, M. Gautschi, A. Pullini, P. D. Schiavone, D. Rossi, and L. Benini, "PULPino: An Open-Source Microcontroller System-on-Chip," in *International Workshop on Power and Timing Modeling, Optimization and Simulation (PATMOS)*, 2016.

[24] F. Bossen, "Common test conditions and software reference configurations," in *JCTVC-L1100*, vol. 12, 2013, p. 7. [Online]. Available: http://phenix.it-sudparis.eu/jct/doc_end_user/current_document.php?id=7281